EXPERIMENT 8

Name: Trimann Kaur
Branch: B.E. CSE (AIML)
Semester: 5
Subject Name: ADBMS

UID: 24BAI70511
Section: 24AIT_KRG-1_B
Date of Performance:07.09.2026
Subject Code: 24CSP-310

1. A company maintains customer information, product information, and sales order information in three separate tables: customer_master, product_catalog, and sales_orders. The company wants to provide an order report to an external client. The report must contain the **Order ID, Order Date, Customer Name, Product Name, and Final Amount**.

For every order, first calculate the **Gross Amount** using:
Gross Amount = Unit Price × Quantity
Then calculate the **Discount Amount** using:
Discount Amount = Gross Amount × Discount Percentage / 100
Finally, calculate the **Final Amount** using:
Final Amount = Gross Amount − Discount Amount

Create a **complex view** using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.

The company also wants to provide access to an external client. Create a PostgreSQL user named client_user and give this user **only SELECT permission on the view**. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

customer_id	full_name	city
C1	Amit Sharma	Delhi
C2	Priya Verma	Mumbai
C3	Ravi Kumar	Bangalore
C4	Neha Singh	Kolkata
C5	Arjun Mehta	Hyderabad

customer_master

product_id	product_name	unit_price
P1	Smartphone X100	25000
P2	Laptop Pro 15	65000
P3	Wireless Earbuds	5000
P4	Smartwatch Fit	30000
P5	Gaming Console	45000

product_catalog

order_id	product_id	quantity	customer_id	discount_percent	order_date
1	P1	2	C1	5	2025-09-01
2	P2	1	C2	10	2025-09-02
3	P3	3	C3	0	2025-09-03
4	P4	1	C4	8	2025-09-04
5	P5	1	C5	12	2025-09-05
6	P1	1	C2	5	2025-09-06
7	P3	2	C1	0	2025-09-07

sales_orders

order_id	order_date	full_name	product_name	final_amount
1	2025-09-01	Amit Sharma	Smartphone X100	47500.00
2	2025-09-02	Priya Verma	Laptop Pro 15	58500.00
3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.00
4	2025-09-04	Neha Singh	Smartwatch Fit	27600.00
5	2025-09-05	Arjun Mehta	Gaming Console	39600.00
6	2025-09-06	Priya Verma	Smartphone X100	23750.00
7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.00

Expected Output – In the Form of a View

Solution:

```
-- CREATING A COMPLEX VIEW
CREATE OR REPLACE VIEW vW_FINAL_AMOUNT_CALCULATION
AS
SELECT S.ORDER_ID, S.ORDER_DATE, C.FULL_NAME, P.PRODUCT_NAME,
(
        ROUND((P.UNIT_PRICE * S.QUANTITY)
        -
        ((P.UNIT_PRICE * S.QUANTITY) * (S.DISCOUNT_PERCENT) / 100), 2)
) AS FINAL_AMOUNT
FROM CUSTOMER_MASTER AS C
JOIN
SALES_ORDERS AS S
ON C.CUSTOMER_ID = S.CUSTOMER_ID
JOIN
PRODUCT_CATALOG AS P
ON P.PRODUCT_ID = S.PRODUCT_ID;

-- CREATING A NEW USER
CREATE USER CLIENT_USER
PASSWORD '12345';

-- GRANTING SELECT PERMISSION TO CLIENT_USER
GRANT SELECT ON vW_FINAL_AMOUNT_CALCULATION TO CLIENT_USER;
```

2. An e-commerce company wants to regularly analyze the performance of its **product categories**. The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple

JOINs, GROUP BY, aggregate functions, DISTINCT counting, calculated values, and CASE statements.

Since the same report is generated frequently, executing the complete analytical query every time can increase database workload. Create a PostgreSQL **Materialized View** named: mv_category_sales

Also demonstrate that when new order data is inserted, the Materialized View does **not automatically reflect the change** and must be refreshed.

Finally, use EXPLAIN ANALYZE to compare the performance of the original analytical query with the Materialized View query.

The report must contain:

- Category ID
- Category Name
- Total Products Sold
- Total Orders
- Unique Customers
- Total Revenue
- Average Revenue per Order
- Category Performance

Business Rules

Requirement	Rule
Total Products Sold	`SUM(quantity)`
Total Orders	`COUNT(DISTINCT order_id)`
Unique Customers	`COUNT(DISTINCT customer_id)`
Total Revenue	`SUM(quantity × unit_price)`
Average Revenue per Order	`Total Revenue / Total Orders`
Excellent	Revenue ≥ 10,000
Good	Revenue ≥ 6,000 and < 10,000
Average	Revenue ≥ 3,000 and < 6,000
Low	Revenue < 3,000

Tasks

Task	Description
1	Create the required source tables.
2	Insert the given category, product, customer, and order data.
3	Write a query to generate the category-wise sales report.
4	Execute the query using `EXPLAIN ANALYZE`.
5	Create the Materialized View `mv_category_sales`.
6	Display the data from the Materialized View.
7	Execute `EXPLAIN ANALYZE` on the Materialized View query.
8	Compare the execution time of both queries.
9	Insert a new order and order detail.
10	Query the Materialized View before refreshing it and observe the stale data.
11	Refresh the Materialized View.
12	Query the Materialized View again and verify the updated result.

category_master

category_id	category_name
C1	Electronics
C2	Clothing
C3	Books

product_master

product_id	product_name	category_id
P101	Laptop	C1
P102	Headphones	C1
P201	Jacket	C2
P202	Shoes	C2
P301	SQL Book	C3

customer_master

customer_id	customer_name
U1	Aman
U2	Priya
U3	Rahul
U4	Simran

order_details

order_id	customer_id	product_id	quantity	unit_price
O101	U1	P101	1	5000
O102	U2	P102	2	1000
O103	U1	P201	1	3000
O104	U3	P202	2	2000
O105	U4	P301	3	500
O106	U2	P101	1	5000
O107	U3	P201	1	3000
O108	U4	P102	1	1000

Output

category_id	category_name	total_products_sold	total_orders	unique_customers	total_revenue	avg_revenue_per_order	category_performance
C1	Electronics	5	4	3	12000	3000.00	Excellent
C2	Clothing	4	3	2	10000	3333.33	Excellent
C3	Books	3	1	1	1500	1500.00	Low

Solution:

```
CREATE MATERIALIZED VIEW MV_CATEGORY_SALES
AS
SELECT
C.CATEGORY_ID, C.CATEGORY_NAME,
SUM(O.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
COUNT(DISTINCT O.CUSTOMER_ID) AS UNIQUE_CUSTOMERS,
(SUM(O.QUANTITY * O.UNIT_PRICE)) AS TOTAL_REVENUE,
ROUND(((SUM(O.QUANTITY * O.UNIT_PRICE)) / COUNT(DISTINCT O.ORDER_ID))
, 2) AS AVERAGE_REVENUE_PER_ORDER,
(CASE WHEN (SUM(O.QUANTITY * O.UNIT_PRICE)) >= 10000 THEN 'EXCELLENT'
        WHEN (SUM(O.QUANTITY * O.UNIT_PRICE)) >= 6000 AND (SUM(O.QUANTITY
* O.UNIT_PRICE)) < 10000 THEN 'GOOD'
        WHEN (SUM(O.QUANTITY * O.UNIT_PRICE)) >= 3000 AND (SUM(O.QUANTITY
* O.UNIT_PRICE)) < 6000 THEN 'AVERAGE'
        WHEN (SUM(O.QUANTITY * O.UNIT_PRICE)) < 3000 THEN 'LOW'
        END
) AS CATEGORY_PERFORMANCE
FROM CATEGORY_MASTER AS C
JOIN PRODUCT_MASTER AS P
ON C.CATEGORY_ID = P.CATEGORY_ID
```

```
JOIN ORDER_DETAILS AS O
ON O.PRODUCT_ID = P.PRODUCT_ID
GROUP BY
   C.CATEGORY_ID,
   C.CATEGORY_NAME;

SELECT * FROM MV_CATEGORY_SALES;

EXPLAIN ANALYZE
SELECT
C.CATEGORY_ID, C.CATEGORY_NAME,
SUM(O.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
COUNT(DISTINCT O.CUSTOMER_ID) AS UNIQUE_CUSTOMERS,
(SUM(O.QUANTITY * O.UNIT_PRICE)) AS TOTAL_REVENUE,
ROUND(((SUM(O.QUANTITY * O.UNIT_PRICE)) / COUNT(DISTINCT O.ORDER_ID))
, 2) AS AVERAGE_REVENUE_PER_ORDER,
(CASE WHEN (SUM(O.QUANTITY * O.UNIT_PRICE)) >= 10000 THEN 'EXCELLENT'
        WHEN (SUM(O.QUANTITY * O.UNIT_PRICE)) >= 6000 AND (SUM(O.QUANTITY
* O.UNIT_PRICE)) < 10000 THEN 'GOOD'
        WHEN (SUM(O.QUANTITY * O.UNIT_PRICE)) >= 3000 AND (SUM(O.QUANTITY
* O.UNIT_PRICE)) < 6000 THEN 'AVERAGE'
        WHEN (SUM(O.QUANTITY * O.UNIT_PRICE)) < 3000 THEN 'LOW'
        END
) AS CATEGORY_PERFORMANCE

FROM CATEGORY_MASTER AS C
JOIN PRODUCT_MASTER AS P
ON C.CATEGORY_ID = P.CATEGORY_ID
JOIN ORDER_DETAILS AS O
ON O.PRODUCT_ID = P.PRODUCT_ID
GROUP BY
   C.CATEGORY_ID,
   C.CATEGORY_NAME;

EXPLAIN ANALYZE
SELECT * FROM MV_CATEGORY_SALES;

INSERT INTO ORDER_DETAILS
VALUES ('O109', 'U2', 'P301', 10, 500);

REFRESH MATERIALIZED VIEW MV_CATEGORY_SALES;
SELECT * FROM MV_CATEGORY_SALES;
```